

Atelier TP — Soirée 15

Duck typing et classes abstraites — la hiérarchie `ModeLivraison`

Cet atelier introduit deux mécanismes complémentaires : le **duck typing** (contrat implicite) et les **classes abstraites** (contrat explicite, `abc.ABC`). Le domaine est neuf et indépendant du fil rouge `Livre` : une famille de modes de livraison d'un colis. Travailler individuellement.

Renommer le squelette `mode_livraison_squelette.py` en `mode_livraison.py` avant de le compléter : on ne modifie pas les fichiers distribués. Le fichier de tests `test_mode_livraison.py` est à créer *ex nihilo* à l'exercice 4 ; ce n'est pas un squelette fourni. Respecter PEP 8 (`PascalCase` pour les classes, `snake_case` pour les méthodes et attributs) et documenter chaque classe et chaque méthode par une docstring au format Google.

Fichiers distribués et fichiers à remettre

Deux fichiers sont remis en début de séance :

- `mode_livraison_squelette.py` : la classe abstraite `ModeLivraison` fournie en entier (à ne pas modifier), et les sous-classes à compléter.
- `mode_livraison_buggy.py` : un fichier autonome servant au diagnostic de l'exercice 1.

Trois fichiers sont à remettre en fin de séance :

- `mode_livraison.py` : le squelette renommé et complété (exercices 2 et 3).
- `test_mode_livraison.py` : la suite de tests créée *ex nihilo* (exercice 4).
- `reponses.txt` : les justifications écrites (exercices 3 et 5).

1. Exercice 1 — Diagnostic : le contrat manquant (AA3)

Le fichier `mode_livraison_buggy.py` contient une tentative d'ajout d'un mode de livraison express premium qui refuse de s'exécuter.

1. Exécuter le fichier tel quel. Relever le message d'erreur exact et préciser à quel moment l'erreur survient : à la *construction* de l'objet, ou au premier *appel* d'une méthode ?
2. Identifier ce qui, dans la classe `LivraisonColisExpress`, provoque ce refus. Expliquer en une à deux phrases pourquoi la classe ne peut pas être instanciée.
3. Nommer le mécanisme en jeu : que devient une sous-classe d'une classe abstraite lorsqu'elle n'implémente pas toutes les méthodes décorées par `@abstractmethod` ?
4. Opposer ce diagnostic au duck typing. Si `LivraisonColisExpress` n'héritait pas de `ModeLivraison` mais était seulement passée à une fonction qui appelle `delai_estime()`, à quel moment l'erreur surgirait-elle ? Indiquer pourquoi la détection à la construction est préférable.

5. Corriger le fichier de sorte que l'instanciation réussisse, puis vérifier que `cout(3.0)` renvoie bien la valeur attendue.

- ✓ Le symptôme est décrit précisément : nature de l'erreur et moment où elle survient.
- ✓ La cause est localisée dans le code de la classe fautive.
- ✓ Le mécanisme propre aux classes abstraites est nommé correctement.
- ✓ L'opposition avec une détection par duck typing est explicitée (à quel moment l'erreur surviendrait sans héritage).
- ✓ La correction proposée rend la classe instanciable, vérifiée par exécution.

2. Exercice 2 — Implémenter la hiérarchie ModeLivraison (AA1)

Renommer `mode_livraison_squelette.py` en `mode_livraison.py`. La classe abstraite `ModeLivraison` est fournie : ne pas la modifier. Compléter les trois sous-classes concrètes en respectant les spécifications suivantes.

1. `LivraisonStandard` (sans `__init__` propre, tarifs en constantes de classe) :
 - `cout(poids_kg)` : valider d'abord le poids via `self._valider_poids(poids_kg)`, puis renvoyer `TARIF_BASE + TARIF_PAR_KG * poids_kg`.
 - `delai_estime()` : renvoyer `DELAI_JOURS` (3).
2. `LivraisonExpress` (état configurable) :
 - `__init__(self, supplement=10.0)` : appeler `super().__init__()`, valider `supplement` (nombre, ni `bool` ; valeur positive ou nulle) en levant `TypeError` ou `ValueError`, puis stocker la valeur.
 - Exposer `supplement` en property **lecture seule**.
 - `cout(poids_kg)` : barème standard augmenté du supplément.
 - `delai_estime()` : renvoyer 1.
3. `PointRelais` (état configurable) :
 - `__init__(self, nom_reseau)` : appeler `super().__init__()`, valider `nom_reseau` (chaîne non vide) en levant `TypeError` ou `ValueError`, puis stocker la valeur.
 - Exposer `nom_reseau` en property **lecture seule**.
 - `cout(poids_kg)` : valider le poids, puis renvoyer le forfait `TARIF_FORFAIT` (indépendant du poids).
 - `delai_estime()` : renvoyer 4.

Vérifier le travail en instanciant les trois modes et en appelant `recapitulatif(2.5)` sur chacun (méthode concrète héritée de l'ABC, à ne pas réécrire).

- ✓ Les trois sous-classes héritent de `ModeLivraison` et sont instanciables.
- ✓ `cout` et `delai_estime` sont implémentées dans chacune, avec docstrings.
- ✓ `LivraisonExpress` et `PointRelais` valident leur état dans `__init__` et l'exposent en property lecture seule.
- ✓ La validation du poids est déléguée à `_valider_poids` et lève `TypeError` ou `ValueError` selon le cas.
- ✓ `recapitulatif(2.5)` produit une chaîne correcte pour chaque mode sans avoir été réécrite.

3. Exercice 3 — Le duck typing en action (AA1 et AA3)

Toujours dans `mode_livraison.py`.

1. Implémenter `RetraitMagasin` sans aucun héritage (ce n'est pas une sous-classe de `ModeLivraison`) : `cout(poids_kg)` valide le poids puis renvoie `0.0` ; `delai_estime()` renvoie `0`.
2. Implémenter `comparer_livraisons(modes, poids_kg)` : la fonction parcourt `modes`, n'utilise que `cout(poids_kg)` et `delai_estime()` (aucun `isinstance`), et renvoie une chaîne d'une ligne par mode.
3. Construire une liste mêlant les trois sous-classes et un `RetraitMagasin`, puis afficher le résultat de `comparer_livraisons` pour un colis de 2,5 kg. La sortie attendue est :

```
LivraisonStandard : 8.74 EUR en 3 jour(s)
LivraisonExpress : 20.74 EUR en 1 jour(s)
PointRelais : 3.50 EUR en 4 jour(s)
RetraitMagasin : 0.00 EUR en 0 jour(s)
```

4. Dans l'interpréteur, comparer les deux résultats suivants et constater que `RetraitMagasin` ne possède pas la méthode `recapitulatif` :

```
>>> isinstance(LivraisonStandard(), ModeLivraison)
True
>>> isinstance(RetraitMagasin(), ModeLivraison)
False
```

5. Dans `reponses.txt`, expliquer en une à deux phrases pourquoi `RetraitMagasin` est accepté par `comparer_livraisons` alors qu'il n'est pas un `ModeLivraison`.

- ✓ `RetraitMagasin` n'hérite pas de `ModeLivraison` mais expose `cout` et `delai_estime`.
- ✓ `comparer_livraisons` ne contient aucun `isinstance` et traite tous les objets de la même manière.
- ✓ La sortie correspond exactement à celle attendue, intrus compris.
- ✓ La justification écrite distingue correctement protocole et contrat.

4. Exercice 4 — Suite de tests (AA2)

Créer `test_mode_livraison.py` *ex nihilo* (aucun squelette n'est fourni pour ce fichier). Importer les classes et la fonction depuis `mode_livraison`. Organiser les tests en classes thématiques `unittest` et conserver le bloc `if __name__ == "__main__": unittest.main()`.

Couvrir au minimum les cas suivants :

- **Contrat** : `ModeLivraison()` lève `TypeError` ; une sous-classe volontairement incomplète, définie à l'intérieur du test, reste non instanciable.
- **Coût** : valeurs attendues, par exemple `LivraisonStandard().cout(2.0)` vaut `7.99` ; validation du poids (`TypeError` pour un type incorrect, `ValueError` pour une valeur négative ou nulle).
- **Délai** : 3 pour `LivraisonStandard` , 1 pour `LivraisonExpress` , 4 pour `PointRelais` .
- **État configurable** : property `supplement` et `nom_reseau` en lecture seule ; rejet d'un supplément négatif et d'un nom de réseau vide.
- **Polymorphisme** : `comparer_livraisons` sur les trois sous-classes produit trois lignes.
- **Duck typing** : `RetraitMagasin` est accepté par `comparer_livraisons` ; un `assertNotIsInstance` confirme qu'il n'est pas un `ModeLivraison` .

- ✓ Le fichier s'exécute sans erreur et tous les tests passent au vert.
- ✓ Les tests sont regroupés en classes thématiques cohérentes.
- ✓ La non-instanciabilité de l'ABC *et* d'une sous-classe incomplète est testée.
- ✓ Les coûts, délais, validations et propriétés en lecture seule sont couverts.
- ✓ Le polymorphisme et le duck typing font chacun l'objet d'au moins un test.

5. Exercice 5 — Justification de conception (AA3)

Rédiger les réponses suivantes dans `reponses.txt` (justifications écrites uniquement).

1. Duck typing ou classe abstraite : indiquer une situation où le duck typing est préférable et une situation où une classe abstraite l'est. Donner un argument pour chaque.
2. Une boutique décide : « nous ne livrons aucun colis de plus de 30 kg ». Discuter au moins deux endroits où cette contrainte pourrait être placée (par exemple dans `cout` de chaque mode, dans la classe abstraite, ou dans le code client qui prépare la commande), puis trancher en justifiant. Préciser si placer cette limite dans une sous-classe de `ModeLivraison` respecterait le principe de substitution vu en S14.
3. Expliquer pourquoi `recapitulatif` est définie une seule fois dans la classe abstraite plutôt que recopiée dans chaque sous-classe. Nommer le bénéfice obtenu.

- ✓ Le choix duck typing contre classe abstraite est argumenté dans les deux sens, sans réponse unique présentée comme absolue.
- ✓ La contrainte des 30 kg est discutée à au moins deux emplacements, avec une décision justifiée.
- ✓ Le lien avec la substitution de Liskov est correctement mobilisé.
- ✓ Le bénéfice de la factorisation de `recapitulatif` dans la classe abstraite est nommé.

Déposer `mode_livraison.py`, `test_mode_livraison.py` et `reponses.txt`. Les corrigés commentés sont distribués en fin de séance.